Львівський національний університет імені Івана Франка · Факультет електроніки та комп'ютерних технологій · Кафедра системного проєктування

Вибрані розділи науки про дані · 2026–2027 н. р.

Лабораторна робота 12

Інтегрована система: RAG + граф знань + API

- **Тема:** ЛР 12. Інтегрована система: RAG + граф знань + API
- **Лекція:** 12-й тиждень, лекція 12 «Наука про дані в роздрібній торгівлі та фінансах»
- **Тривалість:** 2 год аудиторно + ≈ 4 год самостійної підготовки й оформлення
- **Оцінювання:** максимум 2 бали = 5 результатів × 0.3 бала (разом 1.5) + 0.5 бала за захист (5 запитань × 0.1)
- **Термін здачі та захисту:** до кінця 13-го тижня
- **Інструменти:** FastAPI, Neo4j, FAISS
- **Дані / вхід:** результати ЛР 9–11: Neo4j-граф, векторний індекс, RAG-код; FastAPI + uvicorn

1. Мета роботи

Інтегрувати LLM, векторну базу та граф знань в єдину систему з REST API й простим веб-інтерфейсом.

2. Які навички ви наберете

- проєктувати архітектуру з межами довіри та потоками даних;
- поєднувати векторний і графовий пошук у контексті для LLM;
- створювати REST API з валідацією, помилками й журналюванням;
- готувати відтворювані демонстрації системи.

3. Підготовка

- Опрацюйте лекції 11–12 та джерела [13, 16].
- Встановіть fastapi, uvicorn, pydantic, streamlit (або простий HTML).
- Переконайтеся, що Neo4j і індекс запускаються з нуля.

4. Порядок виконання

Робота складається з п'яти результатів. Кожен результат — самостійна частина (таблиця, схема, алгоритм, модель, код, метрики тощо) і оцінюється окремо: 0.3 бала — виконано повністю; 0.15 — виконано частково; 0 — не виконано. Виконуйте результати послідовно: кожен наступний використовує попередній.

Результат 1. Архітектура системи

Крок 1. Накресліть схему: клієнт → API → (векторна БД, Neo4j, LLM) → відповідь; позначте межі довіри та місця зберігання секретів.

Крок 2. Опишіть потоки даних і формати повідомлень.

Крок 3. Складіть перелік ризиків (ін'єкції, витік, відмова сервісів) і заходів.

Крок 4. Запишіть вимоги: затримка, обсяг, права доступу.

Що зараховується. Повністю (0.3 бала): схема з межами довіри, потоки, ризики й заходи. Частково (0.15 бала): схема без меж довіри або ризиків. Інакше — 0.

Результат 2. Інтеграція векторного та графового контексту

Крок 1. Реалізуйте функцію vector_context(q): top-k фрагментів із метаданими.

Крок 2. Реалізуйте graph_context(q): виділіть сутності в запиті й розширте їх сусідами/шляхами в Neo4j (глибина ≤ 2).

Крок 3. Злийте контексти (дедуплікація, обмеження за токенами) і сформуйте промпт із розділами «Фрагменти» та «Зв'язки графа».

Крок 4. Продемонструйте багатокрокове питання, де граф змінює відповідь порівняно з чистим RAG.

Що зараховується. Повністю (0.3 бала): обидва джерела впливають, є приклад багатокрокового питання з порівнянням. Частково (0.15 бала): використано лише одне джерело. Інакше — 0.

Результат 3. REST API

Крок 1. Створіть FastAPI-застосунок з ендпоінтами POST /ask, GET /health, GET /sources/{id}.

Крок 2. Опишіть моделі запиту/відповіді Pydantic; додайте обмеження довжини запиту.

Крок 3. Реалізуйте обробку помилок (таймаут LLM, недоступна БД) із зрозумілими кодами.

Крок 4. Додайте структуроване журналювання (ідентифікатор запиту, тривалість, кількість токенів) без персональних даних.

Крок 5. Перевірте API через /docs (OpenAPI) і curl.

Що зараховується. Повністю (0.3 бала): усі ендпоінти, валідація, обробка помилок, журнали. Частково (0.15 бала): немає валідації або журналювання. Інакше — 0.

Результат 4. Веб-інтерфейс

Крок 1. Створіть інтерфейс (Streamlit або HTML-форма) для введення питання.

Крок 2. Показуйте відповідь, список джерел із цитатами та підграф/зв'язки, використані для відповіді.

Крок 3. Додайте індикатор завантаження та повідомлення про помилки.

Крок 4. Зробіть скріншоти для 3 сценаріїв (простий, багатокроковий, без відповіді).

Що зараховується. Повністю (0.3 бала): відповідь, джерела й зв'язки, помилки, 3 сценарії. Частково (0.15 бала): показано лише відповідь без джерел. Інакше — 0.

Результат 5. Запуск, тести та демонстрація

Крок 1. Напишіть щонайменше 5 автоматичних тестів API (pytest + TestClient): успіх, помилкові дані, відмова, довгий ввід, формат відповіді.

Крок 2. Підготуйте README: залежності, змінні середовища, команди запуску, приклади запитів.

Крок 3. Запишіть коротке відео/GIF (1–2 хв) або скріншоти роботи.

Крок 4. Перевірте запуск «з нуля» на чистому середовищі.

Крок 5. Заповніть декларацію ШІ.

Що зараховується. Повністю (0.3 бала): ≥ 5 тестів, README, демонстрація, запуск з нуля підтверджено. Частково (0.15 бала): тести або відтворюваність запуску відсутні. Інакше — 0.

5. Вимоги до звіту та файлів

- Звіт за шаблоном «Шаблон звіту до лабораторної роботи»; для кожного результату — розділ із описом кроків, фрагментами коду, таблицями, рисунками та висновком.
- Код (ноутбук/скрипти) запускається з нуля; є README (версії, залежності, фіксований seed, інструкція запуску).
- Файли: ЛР12_{Прізвище}_{група}.pdf та .zip; ключі API, паролі, персональні дані не включати.
- Здача в Moodle до терміну; захист — усно на лабораторному занятті.

6. Оцінювання

- **Результати 1–5:** по 0.3 бала (максимум 1.5): повністю — 0.3; частково — 0.15; не виконано — 0. Умови наведено під кожним результатом.
- **Захист і опитування:** 0.5 бала (5 запитань по 0.1 бала); студент також має пояснити будь-яку частину роботи й за вимогою змінити її «наживо».
- **Запізнення** (до закриття модуля): бали за результати множаться на 0.8; після закриття модуля робота не приймається, крім поважних причин.
- **Не пояснив роботу:** бали за результати зменшуються вдвічі (до кратних 0.15).
- **Порушення доброчесності** (списування, вигадані дані чи джерела, неоголошене використання ШІ): робота не зараховується.
- **Декларація використання ШІ** обов’язкова (інструмент, призначення, запити, як перевірено); без неї результати не зараховуються.

7. Контрольні питання (захист)

1. Яка роль графа у GraphRAG?
2. Як ви захищаєте API від некоректних вхідних даних?
3. Що логуєте та чому не логуєте персональні дані?
4. Які межі довіри у вашій системі?
5. Як змінилась би архітектура за 100 запитів на секунду?

8. Академічна доброчесність і використання ШІ

Робота виконується самостійно. Генеративний ШІ дозволено як допоміжний інструмент за умови декларації й розуміння кожної частини роботи. Списування, вигадані дані чи джерела, неоголошене використання ШІ — порушення доброчесності.

9. Рекомендовані джерела

- [13] Lewis; [16] Kleppmann; документація FastAPI: https://fastapi.tiangolo.com